

Software Development Methodology

How the AgingApprentice views product development

The Aging Apprentice software product development methodology is meant to accommodate individuals and teams that have non committed time to work on unfamiliar tasks with unfamiliar tools and technology. While not being overly prescriptive or formal, a structured approach to development is needed to allow for long periods of time between work sessions and still being able to move things forward. The most pervasive framework for software development today is the Agile methodology. Since most code management systems, including the one we use, GitHub, comes with built in project management tools based on this methodology, it makes sense to manage our Github projects as Agile projects so that we can leverage existing tools and reduce project overhead.

What is Agile Development?

Agile development is a set of principles and practices for software development (and other projects) that emphasizes flexibility, collaboration, and iterative progress. It's based on the Agile Manifesto, which values:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a fixed plan

In practice, Agile breaks work into smaller cycles (iterations), focuses on delivering usable results frequently, and adapts to feedback along the way. Scrum and Kanban, which we discussed earlier, are two popular frameworks under the Agile umbrella.

Key Features of Agile

- Iterative: Deliver small, functional pieces of work regularly (e.g., every 1-4 weeks).
- Collaborative: Teams work closely together and with stakeholders to refine goals.
- Adaptive: Plans evolve based on feedback and changing needs.
- Customer-Focused: Prioritize what delivers value to the end user.

Is Agile a Good Approach for Volunteer Teams?

Volunteer teams—like our rag-tag group of non-dedicated, non-expert developers—have unique dynamics: sporadic availability, varying skill levels, and often intrinsic motivation rather than financial incentives. Let's weigh Agile's fit for such teams.

Pros of Agile for Volunteer Teams

- **Flexibility:** Agile's focus on adapting to change suits volunteers whose schedules fluctuate. Kanban, in particular (as recommended earlier), lets people pick up tasks when they're available.
- **Small Wins:** Delivering bite-sized results keeps motivation high—crucial for volunteers who thrive on seeing impact without long-term commitments.
- **Collaboration:** Agile encourages communication, which can build camaraderie and help less-experienced volunteers learn from peers.
- **Prioritization:** With limited time, Agile's emphasis on tackling high-value tasks first ensures effort isn't wasted—a big plus for volunteers juggling other priorities.

Cons of Agile for Volunteer Teams

- **Time Commitment:** Some Agile practices (e.g., Scrum's daily stand-ups or sprint planning) might strain volunteers who can't commit consistently. This is less of an issue with Kanban's looser structure.
- **Learning Curve:** Agile concepts like iterations, backlogs, or velocity might confuse non-experts unfamiliar with project management. Simplified versions are needed.
- **Coordination Overhead:** Volunteers may resist structured processes, preferring autonomy. Too many meetings or rules could dampen enthusiasm.
- **Uneven Participation:** Agile assumes a team effort, but if some volunteers contribute more than others, it can lead to frustration or bottlenecks.

Making Agile Work for Volunteers

Agile can absolutely work for volunteer teams if tailored to their context:

- **Keep it Simple:** Use Kanban over Scrum to avoid rigid cycles and minimize meetings. A shared board (e.g., Trello) with “To Do,” “Doing,” and “Done” columns is often enough.
- **Short Cycles:** If using iterations (like Scrum sprints), keep them brief—1 week max—to match volunteer availability and maintain momentum.
- **Low Process Overhead:** Skip formal roles (e.g., Scrum Master) and focus on self-organization. Limit meetings to quick check-ins only when needed.
- **Clear Goals:** Ensure tasks are well-defined and meaningful, so volunteers feel their time matters. Non-experts benefit from explicit instructions.
- **Celebrate Progress:** Highlight completed work to sustain morale—volunteers often stick around when they see tangible results.

Is It a Good Approach?

Yes, with caveats. Agile is a great fit for volunteer teams if you lean toward its lighter, more flexible flavours (like Kanban) and avoid overburdening the group with process. For your non-dedicated, non-expert crew, Agile's adaptability aligns well with their constraints—far better than rigid traditional methods like Waterfall, which demand long-term planning and full availability.

Compared to no methodology, Agile provides just enough structure to keep things moving without stifling the volunteer spirit. I'd recommend starting with a basic Kanban setup: a visual board, a handful of clear tasks, and a WIP limit (e.g., 2 tasks per person). Test it for a month, then tweak based on what your team finds helpful.

Scrum vs. Kanban Agile Framework

Let's break down the difference between Scrum and Kanban, and then figure out which might work better for a group of non-dedicated, non-expert developers.

Scrum

Scrum is a structured framework within Agile methodology, designed to manage complex projects with a focus on teamwork, iteration, and adaptability. Here's how it works:

- **Roles:** It has defined roles—Product Owner (prioritizes work), Scrum Master (facilitates the process), and the Development Team (does the work).
- **Sprints:** Work is divided into fixed-length cycles (usually 2-4 weeks) called sprints. Each sprint starts with planning (what to do) and ends with a review (what was done) and retrospective (how to improve).
- **Meetings:** Regular events like daily stand-ups (quick progress check-ins), sprint planning, reviews, and retrospectives are baked in.
- **Focus:** Deliver a potentially shippable product increment at the end of each sprint.
- **Change Management:** Changes mid-sprint are discouraged to maintain focus.

Scrum is great when you need discipline, clear goals, and a rhythm to keep everyone aligned. But it requires commitment—everyone needs to buy into the process, show up for meetings, and stick to the sprint plan.

Kanban

Kanban is a more flexible, visual approach to workflow management, also rooted in Agile principles but less rigid than Scrum. Here's the gist:

- **Board:** Work is visualized on a Kanban board with columns like “To Do,” “In Progress,” and “Done.” Each task is a card that moves across the board.
- **No Fixed Cycles:** Unlike Scrum's sprints, Kanban is continuous—you work on tasks as they come and finish them at your own pace.
- **Work-in-Progress (WIP) Limits:** You limit how many tasks can be “In Progress” at once (e.g., max 3) to avoid overloading the team.
- **Focus:** Optimize flow and reduce bottlenecks by tracking how work moves through the system.
- **Change Management:** New tasks can be added anytime, as long as WIP limits allow.

Kanban shines when you want simplicity, flexibility, and a way to see progress without enforcing a strict schedule or roles.

Key Differences

Aspect	Scrum	Kanban
--------	-------	--------

Structure	Prescriptive (roles, sprints)	Flexible (no roles, no sprints)
Time-boxing	Fixed sprints (e.g., 2 weeks)	Continuous flow
Meetings	Required (daily stand-ups, etc.)	Optional
Change Handling	Mid-sprint changes avoided	Changes anytime, within WIP
Best For	Teams needing structure	Teams valuing flexibility

Which is Better for Non-Dedicated, Non-Expert Developers?

Your group isn't fully dedicated (likely juggling other responsibilities) and lacks deep expertise, so let's weigh the fit:

Scrum Challenges

- Time Commitment: Sprints and mandatory meetings (daily stand-ups, planning, etc.) might overwhelm a part-time team. Non-dedicated folks may struggle to attend consistently.
- Learning Curve: Scrum's roles and ceremonies (like sprint planning or retrospectives) can feel complex for non-experts who aren't familiar with Agile jargon or processes.
- Rigidity: If priorities shift mid-sprint (common with less experienced teams), Scrum's "no changes" rule could frustrate them.

Kanban Advantages

- Simplicity: A Kanban board is intuitive—tasks move from "To Do" to "Done." Non-experts can grasp it quickly without needing to learn a framework.
- Flexibility: No fixed sprints mean the team can adapt to interruptions or shifting priorities, which suits a non-dedicated group.
- Low Overhead: No required meetings or roles—just set up a board, agree on WIP limits, and start. Less pressure on busy or inexperienced developers.
- Visibility: Everyone sees what's being worked on, which helps a less skilled team stay coordinated without heavy process.

Verdict

Kanban is likely better for the Aging Apprentice work environment. It's lightweight and adaptable, letting non-dedicated developers contribute at their own pace without the burden of Scrum's structure. A simple digital board (e.g., Trello or Jira) with columns like "Backlog," "In Progress," and "Done," plus a WIP limit of 2-3 tasks per person, could keep things moving without overwhelming anyone.

That said, if your team craves more guidance and can commit to short sprints (say, 1 week), a stripped-down Scrum could work—focus on just sprints and stand-ups, skipping the full ceremony. But for ease and flexibility, Kanban's the safer bet.